**Автономная некоммерческая организация высшего образования**
**«Университет Иннополис»**
**(АНО ВО «Университет Иннополис»)**

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА**
**(БАКАЛАВРСКАЯ РАБОТА)**
**по направлению подготовки**
**09.03.01 – «Информатика и вычислительная техника»**

**GRADUATION THESIS**
**(BACHELOR'S GRADUATE THESIS)**
**Field of Study**
**09.03.01 – «Computer Science»**

**Направленность (профиль) образовательной программы**
**«Информатика и вычислительная техника»**
**Area of Specialization / Academic Program Title:**
**«Computer Science»**

| | |
|---|---|
| **Тема / Topic** | **Оценка сопровождаемости iOS-кода, созданного ИИ-агентами, и влияние архитектурных правил / Accessing the Maintainability of AI-Generated iOS Code and the Effect of Architecture Rules** |

| | | |
|---|---|---|
| Работу выполнил / Thesis is executed by | **Исупов Александр Андреевич / Isupov Aleksandr Andreevich** | подпись / signature |
| Руководитель выпускной квалификационной работы / Graduation Thesis Supervisor | **Садовых Андрей Александрович / Sadovykh Andrey Aleksandrovich** | подпись / signature |
| Консультанты / Consultants | | подпись / signature |

Иннополис, Innopolis, 2026

# Contents


**1 Introduction** **7**

1.1 Background . . . . . 7

1.2 Problem statement and research gap . . . . . 8

1.3 Purpose and research questions . . . . . 8

1.4 Tasks . . . . . 9

1.5 Approach and definition of done . . . . . 10

1.6 Experimental material and its constraints . . . . . 10

1.7 Scientific novelty . . . . . 11

1.8 Practical significance . . . . . 12

1.9 Structure of the thesis . . . . . 12

**2 Literature Review** **13**

2.1 Two ways to measure maintainability . . . . . 13

2.2 The project as a graph . . . . . 14

2.2.1 Propagation cost . . . . . 14

2.2.2 Instability . . . . . 14

2.2.3 Class-level metrics . . . . . 15

2.3 Architectural smells . . . . . 15

2.4 Limits of the graph model . . . . . 16

2.5 Code written and refactored by AI agents . . . . . 18
2.6 Related work on agents and architecture . . . . . 18
2.7 iOS specifics . . . . . 19
2.8 Rules as instructions and as a checker . . . . . 19
2.9 Conclusion: the gap and the choice of method . . . . . 20

**3 Methodology** **21**

**4 Implementation** **23**

**5 Evaluation and Discussion** **24**

**6 Conclusion** **25**

**Bibliography cited** **26**

**A Extra Stuff** **29**

**B Even More Extra Stuff** **30**

# List of Tables

3.1 This is the title I want to appear in the List of Tables . . . . . 21

# List of Figures

## Abstract

My abstract starts from here.

# Chapter 1

# Introduction

## 1.1 Background

Developers increasingly assign programming tasks to AI coding agents in plain text. The agent reads the project, edits the files and commits the result. This mode of work has become common in industry.

The resulting code usually works. The difficulty appears later, when the project has to be changed. Maintainability denotes the effort required to understand, modify and extend a system. A project can pass its tests today and remain almost impossible to change in a year.

Agent-written code carries a higher risk here. An agent optimises for the task in the prompt. Structure stays outside its objective and outside any build check. Each task adds a file, a class and a dependency. No single step looks wrong, but their sum changes the shape of the project. Cordeiro et al. observe this pattern directly: a large language model renames elements and extracts methods readily, yet attempts structural refactoring rarely and performs it worse than developers [1]. Pearce et al. report a related finding for security: about 40 per cent of the

suggestions produced by GitHub Copilot in security-relevant scenarios contained vulnerabilities [2]. The code passes review, and the structural debt accumulates.

## 1.2 Problem statement and research gap

Quality tools work at the level of a file or a single class. They count lines, measure complexity and record coupling between classes. These measurements are well tested, but they do not see architecture, that is, the way large parts of a project depend on each other. A project can keep every class clean while its structure falls apart, and such tools report nothing.

Swift widens the gap further. No validated instrument for architectural analysis of Swift code was found in the reviewed literature. Existing detectors were built and validated for Java, and the accuracy lost in a transfer to Swift has never been measured.

The second half of the gap concerns the proposed remedy. A common suggestion is to write architecture rules, which state who may depend on whom, and to give them to the agent. The idea sounds reasonable, yet the effect of such rules on the produced code remains unmeasured. No controlled comparison of agent output with and without rules was found for mobile projects.

## 1.3 Purpose and research questions

This thesis aims to build an instrument that measures the architectural maintainability of iOS code produced by an AI coding agent, and to apply the instrument in a controlled experiment on architecture rules.

**RQ1.** Which structural metrics and architectural smells can be computed for an

iOS project, and at what accuracy can they be extracted from Swift source code?

**RQ2.** Does a set of architecture rules given to a coding agent change the structure of the code it produces?

**RQ3.** How much accuracy does the transfer of a Java-validated instrument to Swift cost?

## 1.4 Tasks

The following tasks are solved to answer these questions.

1. Review structural metrics, catalogues of architectural smells and studies of agent-written code, and select those applicable to iOS.
2. Design a measurement profile with different roles and a stated order of trust between the measurements.
3. Extract a component dependency graph from Swift source code, because import analysis does not work inside one module.
4. Build a rule set and a checker for it.
5. Conduct an experiment on open-source iOS projects with and without the rules.
6. Validate the instrument independently of the experiment.

## 1.5 Approach and definition of done

The experiment compares two conditions on the same project, the same starting commit and the same task text: an agent alone and an agent with the rule set. The rule set is identical for every task, so the conditions differ only in its presence. Each task runs several times per condition, because agent output is non-deterministic, and the comparison stays paired within a task.

A task counts as solved under three conditions: the project builds, the existing test suite passes, and hidden acceptance tests taken from the reference pull request pass. Tasks whose reference pull request contains no tests are excluded, since the human solution would otherwise be unverifiable. Regression tests alone cannot confirm that behaviour survived a change. Mutation testing would be stronger, but public iOS projects rarely ship suites prepared for it. The study therefore measures structure with the profile and verifies behaviour with the hidden tests.

Task size is taken from the reference patch, namely the number of changed lines and touched files, mapped to a coarse class of small, medium or large. Historical tasks carry no story points, and this measure stays reproducible.

## 1.6 Experimental material and its constraints

The study draws about 30 tasks from the history of roughly 10 open-source iOS projects. Each task originates from a real issue or a merged pull request, rolled back to the commit that precedes its solution, so a human reference solution always exists. Several tasks per project allow a paired comparison within each task while covering different codebases.

The minimum project size follows from the metrics rather than from con-

venience. A hub-like part needs a dense neighbourhood, a cycle needs at least three parts, and PC changes in steps of $1/N^2$, so small graphs produce unstable values. A further criterion supports project selection: the sample must consist of mature projects whose structure is large enough to degrade in a measurable way. Section **??** derives the lower bound from these constraints and fixes the selection threshold before any run, applying it equally to every candidate project.

One limitation follows from the task source. The task text originates from a closed issue, so an agent with network access could retrieve the published solution. Runs disable network access where the tooling permits, and Chapter **??** discusses the residual risk.

## 1.7 Scientific novelty

1. Architectural smell detection validated on Java is transferred to Swift, and the accuracy lost in the transfer is measured instead of assumed.

2. A measurement profile is proposed in which metrics have different roles and an explicit order of trust, which answers the documented noise of single metrics.

3. The same rule set serves twice, as the agent's input and as the ruler for its output. This separation distinguishes a rule the agent did not know from a rule it broke anyway.

4. First empirical data are obtained on whether architecture rules change the structure of agent-written iOS code.

## 1.8 Practical significance

The profile applies to any iOS project, including human-written ones, and gives a team a short report about the parts that became harder to change. The rule set can be added directly to the instructions given to an agent. This makes the intervention cheap and requires no changes to the agent itself.

## 1.9 Structure of the thesis

The thesis contains an introduction, five chapters, a conclusion and a list of references. Chapter 2 reviews related work. Chapter 3 describes the measurement profile and the design of the tool. Chapter 4 describes graph extraction for Swift and the rule set. Chapter 5 presents the experiment and its results. Chapter **??** discusses the results, the threats to validity and future work.

# Chapter 2

# Literature Review

This review covers more than 30 works, grouped into four areas: structural metrics of software, catalogues of architectural smells, documented limits of graph-based measurement, and studies of code written and refactored by AI agents. The sections below classify each group, evaluate its results and justify the methods chosen for the present study.

## 2.1 Two ways to measure maintainability

Maintainability can be studied from the history of a project, that is, from how often code changes and how many defects appear, or from the structure of the code as it stands today. The first way needs a long commit history, which a project written by an agent does not have. The present work takes the second way, and the review follows the structure of the measurements it uses.

## 2.2 The project as a graph

All structural metrics used here start from one idea. A project is a set of *parts* and a set of *dependencies* between them. A part is not a file but a larger unit, namely a feature or a core subsystem. The result is a directed graph. MacCormack, Rusnak and Baldwin applied this model to open source and proprietary systems and showed that it describes a system well enough for comparison [3].

### 2.2.1 Propagation cost

The first metric shows how far a change can travel through a system [3]:

$$PC = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} V_{ij}, \tag{2.1}$$

where $N$ denotes the number of parts, and $V_{ij}$ equals one if part $i$ reaches part $j$ directly or through a chain of parts, and zero otherwise. The value $N^2$ gives the number of all possible pairs, so $PC$ is a share between zero and one. A high value means that touching one part can affect almost everything else.

### 2.2.2 Instability

The second metric shows how much one part is locked in by the rest of the system [4]:

$$I(C) = \frac{C_e}{C_a + C_e}, \tag{2.2}$$

where $C_a$ denotes the number of parts that depend on $C$, and $C_e$ denotes the number of parts that $C$ depends on. With $I = 0$ many parts rely on $C$ while $C$ relies on nobody, so the part is stable and expensive to change. With $I = 1$ the

situation reverses, and the part is free to change. Neither value is harmful by itself, and the problem appears in the combination (Section 2.3). Hyrynsalmi and Leppänen validated the metric empirically [5].

### 2.2.3 Class-level metrics

The oldest and best-tested family works at the level of a single class. The suite of Chidamber and Kemerer [6] includes CBO, the number of other classes a class uses; LCOM, the degree to which its methods belong together; and WMC, its complexity. Basili, Briand and Melo showed that these metrics predict where defects appear [7]. Their limit lies in the level of detail. They look at one class at a time, so a project can keep clean classes and remain impossible to change. This limit explains why the tools that implement the family stay silent about architecture.

## 2.3 Architectural smells

A different line of work recognises a pattern instead of computing a number. An *architectural smell* denotes a named configuration of dependencies that experience shows to be harmful. Azadi, Fontana and Taibi collected catalogues of such smells for component graphs [8], and the Arcan tool implements their detection [9]. The present work uses four of them:

- a cyclic dependency, where parts depend on each other and cannot be changed separately;
- a god component, which collects most of the code and most of the dependencies;

- a hub-like part, which has many incoming and many outgoing dependencies;
- an unstable dependency, where a stable part depends on a volatile one.

These smells do not form a separate measurement. Each of them is a named reading of the graph numbers from the previous section, and the measurement profile exploits this relation in Chapter 3.

Several studies support the practical value of the patterns. Macia et al. showed that single code anomalies do not predict architectural problems on their own [10]. Sas et al. linked architectural smells to co-changing files [11]. Mo et al. formalised the *hotspot*, a small set of components that attracts most of the changes [12]. Together these results make the four patterns a reasonable proxy for maintenance cost.

## 2.4 Limits of the graph model

**Dead edges.** Geipel and Schweitzer compared the dependency graph of a project with its change history [13]. More than half of all dependencies never took part in a change, and ten per cent of the active ones caused more than half of all propagation. Counting every edge equally therefore overstates coupling. A correction by history requires a long commit log, which the present study does not have.

**Invisible edges.** Ribeiro et al. described *semantic dependencies*, that is, real behavioural coupling that leaves no trace in the code. One type writes values in a certain order, and another type silently assumes that order [14]. A project can compile, pass its tests and still behave incorrectly.

**Uneven shape.** Baldwin, MacCormack and Rusnak found the same structure in many graphs: a small dense core and a loose periphery, with the core deciding the cost of everything [15]. A useful report therefore states that the core grew and that two modules moved into it, not that *PC* rose by three per cent.

**Blindness to good restructuring.** Nord et al. restructured a system into proper layers and found that *PC* did not move at all [16]. Their correction weights each edge by the degree to which the crossed boundary shields changes:

$$PC_w = \frac{1}{N^2} \sum_{i,j} V_{ij}\, w(\ell_{ij}), \tag{2.3}$$

where $\ell_{ij}$ denotes the boundary crossed by the dependency, and $w(\ell)$ stays small if that boundary absorbs changes well.

**Noise.** Hamdi et al. measured 300 Android projects before and after refactoring and found no significant change in most metrics [17]. Maffort et al. built a detector of architectural rule violations that found 96 per cent of the real violations, while only 47 to 89 per cent of its reports were correct [18]. False alarms belong to this kind of analysis and must be expected.

The conclusion drawn here is that a single quality score should be avoided. Several measurements with different roles are required, and the order of trust between them must be stated explicitly.

## 2.5 Code written and refactored by AI agents

Pearce et al. examined code suggested by GitHub Copilot in security-relevant scenarios and found vulnerabilities in about 40 per cent of the suggestions [2]. Cordeiro et al. compared a large language model with human developers on 30 Java projects. The model renamed elements and extracted methods readily, but attempted structural refactoring much less often and performed it worse [1]. If an agent adds code on top of the existing code instead of reshaping it, structure becomes exactly the property that requires supervision.

## 2.6 Related work on agents and architecture

Work that combines agents with software structure falls into two groups. Each group is described below together with what it leaves open.

**Agents as refactoring tools.** RefAgent assembles a pipeline of specialised agents that select refactoring candidates, plan the change, apply it and verify the result [19]. The authors report improvements over single-agent baselines on Java projects. The setting of such work is the refactoring task itself, where the agent receives an explicit instruction to improve structure.

**Empirical studies of refactoring performed by agents.** A recent study of agentic refactoring examines refactoring commits produced by coding agents in open-source repositories and compares them with human refactoring [20]. The reported picture is that agents refactor less often than developers and prefer low-risk transformations. Cordeiro et al. reach a compatible conclusion in a

controlled setting, where structural refactoring appeared rarely in the output of a large language model [1].

Both groups study refactoring as an explicit task, while an agent in normal use receives feature tasks and touches structure as a side effect. Assessment of generated code has so far stayed at the level of defects and duplication (Section 2.5). Neither of these groups measures architecture, and neither covers Swift or iOS. No controlled comparison of agent output with and without architecture rules was found for mobile projects.

## 2.7 iOS specifics

Rahkema and Pagano showed that architectural problems occur in real iOS projects [21]. Automated test coverage in open-source iOS projects stays low, which limits any check of whether behaviour survived a change. UIKit still accounts for the majority of real projects, so one framework provides a reasonable scope [22]. Most importantly, the reviewed literature contains no validated tooling for architectural analysis of Swift code.

## 2.8 Rules as instructions and as a checker

Terra and Valente proposed a dependency constraint language in which architecture rules are written declaratively and checked automatically [23]. Wong et al. detected modularity violations from call relations instead of size or coupling [24]. Ribeiro et al. argued that explicit contracts between parts reduce both syntactic and semantic dependencies [14]. This literature gives rules a double role: they serve as instructions for an agent, and the same rules check the result

automatically.

## 2.9 Conclusion: the gap and the choice of method

The reviewed work provides a mature set of tools, namely the graph model, propagation cost, instability, class-level metrics and catalogues of smells. It also provides a clear warning that single metrics are noisy, blind to history and to semantics, and silent about the shape of a system.

Three gaps follow from this review.

1. No validated instrument for architectural analysis of Swift code exists, and the cost of a transfer from Java remains unknown.

2. The documented noise of single metrics is usually handled by hand; a small profile with explicit roles and an order of trust is missing.

3. The effect of architecture rules on agent output remains unmeasured for ordinary feature tasks.

The choice of method follows from these gaps. Graph-based measurement suits the problem, because it needs no commit history and applies to a project immediately after the agent finishes. The four smells provide a validated vocabulary for reading the graph. The profile form answers the documented noise, and the same rule set answers the third gap by serving both as input and as a measuring instrument. Chapters 3 to 5 develop this method.

# Chapter 3

# Methodology

Referencing other chapters 2, 3, 4, 5 and 6

TABLE 3.1
Simulation Parameters

| A | B |
|---|---|
| **Parameter** | **Value** |
| Number of vehicles | $\mid \mathcal{V} \mid$ |
| Number of RSUs | $\mid \mathcal{U} \mid$ |
| RSU coverage radius | 150 m |
| V2V communication radius | 30 m |
| Smart vehicle antenna height | 1.5 m |
| RSU antenna height | 25 m |
| Smart vehicle maximum speed | $v_{max}$ m/s |
| Smart vehicle minimum speed | $v_{min}$ m/s |
| Common smart vehicle cache capacities | $[50, 100, 150, 200, 250]$ mb |
| Common RSU cache capacities | $[5000, 1000, 1500, 2000, 2500]$ mb |

| A | B |
|---|---|
| Common backhaul rates | $[75, 100, 150]$ mb/s |

...

# Chapter 4

# Implementation

...

# Chapter 5

# Evaluation and Discussion

...

# Chapter 6

# Conclusion

...

# Bibliography cited


[1] J. Cordeiro, S. Noei, and Y. Zou, "An empirical study on the code refactoring capability of large language models," 2024.

[2] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of github copilot's code contributions," in *IEEE Symposium on Security and Privacy*, 2022.

[3] A. MacCormack, J. Rusnak, and C. Baldwin, "Exploring the structure of complex software designs: An empirical study of open source and proprietary code," *Management Science*, 2006.

[4] R. C. Martin, *Object-Oriented Design Quality Metrics: An Analysis of Dependencies*. 1994.

[5] S. Hyrynsalmi and M. Leppänen, "A validation of the instability metric," 2015.

[6] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," *IEEE Transactions on Software Engineering*, 1994.

[7] V. R. Basili, L. C. Briand, and W. L. Melo, "A validation of object-oriented design metrics as quality indicators," *IEEE Transactions on Software Engineering*, 1996.

[8] U. Azadi, F. A. Fontana, and D. Taibi, "Architectural smells detected by tools: A catalogue proposal," in *International Conference on Technical Debt*, 2019.

[9] F. A. Fontana, I. Pigazzini, R. Roveda, and M. Zanoni, "Arcan: A tool for architectural smells detection," in *IEEE International Conference on Software Architecture Workshops*, 2017.

[10] I. Macia, J. Garcia, D. Popescu, A. Garcia, and N. Medvidovic, "Are automatically-detected code anomalies relevant to architectural modularity?" In *Aspect-Oriented Software Development*, 2012.

[11] D. Sas, P. Avgeriou, and F. A. Fontana, "Investigating instability architectural smells evolution," 2019.

[12] R. Mo, Y. Cai, R. Kazman, and L. Xiao, "Hotspot patterns: The formal definition and automatic detection of architecture smells," in *Working IEEE/IFIP Conference on Software Architecture*, 2015.

[13] M. Geipel and F. Schweitzer, "The impact of dependencies on the modularity of software systems," 2015.

[14] M. Ribeiro et al., "On the reduction of semantic dependencies in software architectures," 2014.

[15] C. Baldwin, A. MacCormack, and J. Rusnak, "The architecture of complex systems: Do core-periphery structures dominate?," 2014.

[16] R. L. Nord, I. Ozkaya, R. S. Sangwan, J. Gonzalez, and R. Alexander, "Climbing the stairway to heaven: Architectural design and measurement," 2012.

[17] O. Hamdi, A. Ouni, E. Alomar, and M. W. Mkaouer, "An empirical study on the impact of refactoring on quality metrics in android applications," in *International Conference on Mobile Software Engineering and Systems*, 2021.

[18] C. Maffort, M. T. Valente, R. Terra, M. Bigonha, N. Anquetil, and A. Hora, "Mining architectural violations from version history," in *Working Conference on Mining Software Repositories*, 2013.

[19] F. K. Khouloud Oueslati Maxime Lamothe, "Refagent: A multi-agent llm-based framework for automatic software refactoring," 2025.

[20] K. Horikawa, H. Li, Y. Kashiwa, B. Adams, H. Iida, and A. E. Hassan, "Agentic refactoring: An empirical study of ai coding agents," Nov. 2025.

[21] K. Rahkema and D. Pagano, "Analysis of architectural smells in ios applications," 2020.

[22] L. Baresi et al., "An empirical study of swiftui and uikit adoption in ios applications," 2024.

[23] R. Terra and M. T. Valente, "A dependency constraint language to manage object-oriented software architectures," *Software: Practice and Experience*, 2009.

[24] S. Wong, Y. Cai, M. Kim, and M. Dalton, "Detecting software modularity violations," in *International Conference on Software Engineering*, 2011.

# Appendix A

# Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

# Appendix B

# Even More Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.